
NiCE Text Grid Programmer's Guide Version 20.2.0.2

- [Copyright](#)
- [Document Lifecycle](#)
 - [Document States](#)
- [Document Types](#)
 - [Common Document Fields](#)
 - [Email Documents](#)
 - [Chat Documents](#)
 - [General Documents](#)
- [Languages](#)
- [Error Responses](#)
 - [Global Error Responses](#)
- [Security](#)
 - [HTTPS Support](#)
- [Document Operations](#)
 - [Add/Update Document](#)
 - [Add/Update Document Batch](#)
 - [Get Document](#)
 - [Get Document Batch](#)
 - [Get Tokens](#)
 - [Get Tokens Without Indexing a Document](#)
 - [Delete Document](#)
 - [Delete Document Batch](#)
- [Search Operations](#)
 - [Search](#)
 - [Get Next Scroll](#)
 - [Clear Scroll](#)
- [Other Operations](#)
 - [About](#)
 - [License](#)
 - [Search Metrics](#)
 - [Structured Query Validation](#)

- [Chat File Format](#)
- [Structured Queries](#)
 - [Fuzziness](#)
 - [JSON Envelope](#)
 - [Structured Query Language](#)
- [Source URI Support](#)
 - [File System Source URIs - Windows](#)
 - [File System Source URIs - Linux](#)
 - [File System Source URIs - All Platforms](#)
 - [HTTP Source URIs](#)
- [Changes From Text Grid 1.x to Text Grid 2.x](#)
 - [Document Flow](#)
 - [Get Tokens Support for artificial documents](#)
 - [Synchronous CRUD Calls](#)
 - [Notifications](#)
 - [Document Operations](#)
 - [source_uri field](#)
 - [Email files](#)
 - [Search Wildcards](#)
 - [Get Document Status Batch Has Been Eliminated](#)
 - [Support for Stemming Search has been eliminated](#)
 - [Support for some chat info fields has been removed](#)
 - [Query Validation is more restrictive on Poison Queries by Default](#)

Copyright

PROPRIETARY AND CONFIDENTIAL INFORMATION

Information herein is proprietary information and trade secrets of NICE Ltd. and/or its affiliated companies (Affiliates). This document and the information herein is the exclusive property of NICE and its Affiliates and shall not be disclosed, in whole or in part, to any third party or utilized for any purpose other than the express purpose for which it has been provided.

IMPORTANT NOTICE

Subject always to any existing terms and conditions agreed between you and NICE or any Affiliate with respect to the products which are the subject matter of this document, neither NICE nor any of its Affiliates shall bear any responsibility or liability to a client or to any person or entity with respect to liability, loss or damage caused or alleged to be caused directly or indirectly by any product supplied or any reliance placed on the content of this document. This includes, but is not limited to, any interruption of service, loss of business or anticipatory profits or consequential damage resulting from the use or operation of any products supplied or the content of this document. Information in this document is subject to change without notice and does not represent a commitment on the part of NICE or any Affiliate.

All information included in this document, such as text, graphics, photos, logos and images, is the exclusive property of NICE or in Affiliate and is protected by United States and international copyright laws. Permission is granted to use, view and photocopy (or print) materials from this document only in connection with the products to which this document relates and subject to the terms of license applicable to such products. Any other use, copying, distribution, retransmission or modification of the information in this document without the express prior written permission of NICE or an Affiliate is strictly prohibited. In the event of any permitted copying, redistribution or publication of copyrighted material, no changes in, or deletion of, author attribution, trademark legend or copyright notice shall be made.

Products supplied may be protected by one or more of the US patents listed at www.nice.com/Patents. For the full list of trademarks of NICE and its Affiliates, visit www.nice.com/Nice-Trademarks. All other marks used are the property of their respective proprietors.

All contents of this document are: Copyright © 2025 NICE Ltd. All rights reserved

For assistance, contact your local supplier or nearest NICE Customer Service Center:

**EMEA Region (Europe,
Middle East, Africa)**

Tel: +972-9-775-3800
Fax: +972-9-775-3000
email: support@nice.com

**APAC Region (Asia/
Pacific)**

Tel: +852-8338-9818
Fax: +852-2802-1800
email: support.apac@nice.com

North America

Tel: 1-800-663-5601
Fax: +201-356-2197
email: na_sales@nice.com

International

Headquarters-Israel
Tel: +972-9-775-3100
Fax: +972-9-775-3070
email: info@nice.com

**The Americas Region
(North, Central, South
America)**

Tel: 1-800-6423-611

Fax: +720-264-4012

email:

support.americas@nice.com

Israel

Tel: 09-775-3333

Fax: 09-775-3000

email: support@nice.com

France

Tel: +33-(0)1-41-38-5000

Fax: +33-(0)1-41-38-5001

Hong-Kong

Tel: +852-2598-3838

Fax: +852-2802-1800

United Kingdom

Tel: +44-8707-22-4000

Fax: +44-8707-22-4500

Germany

Tel: +49-(0)-69-97177-0

Fax:

+49-(0)-69-97177-200

NICE invites you to join the NICE User Group (NUG).

Visit the NUG Website at www.niceusergroup.org; and follow the instructions.

All queries, comments, and suggestions are welcome! Please email: nicebooks@nice.com.

For more information about NICE, visit www.nice.com

Document Lifecycle

Text Grid organizes information into *documents* that are the targets of parsing, enriching, and searching. The client application is responsible for adding each document to Text Grid and assigning it an ID. The client may also specify the document's *source artifact*, such as an email or word processing file. Then Text Grid processes the document, which involves (optionally) reading from the source url, and extracting other information from the source artifact and adding it to the document in Text Grid. When processing is complete, Text Grid *indexes* the document, making it available for full-text search.

Document States



Document Types

Text Grid accepts three types of input document, each with its own data structure and requirements for file format: [email](#), [chat](#), and [general](#). While documents differ in their structure, and the details of their processing, all types share a common lifecycle as expressed in their document states.

The client application specifies a *document ID* for each document. The ID uniquely represents each document, even across types. If a programmer does use a single ID in multiple languages or multiple types, then Text Grid does NOT do any checking for this at this point for performance reasons, but it is considered an illegal operation, and Text Grid will later fail if trying to retrieve a document for deletion, get, or get-token using the ALL parameter for language.

Important As of Text Grid 20.1.0, the document ID prefers positive 64 bit integers. Any non-digit alphanumeric value (or integer greater than 64 bits) used for an ID will be produce a deprecation warning. This was done to allow ID range based indices for NIA.

Clients are encouraged to use a natural key for the document's ID value; however, this is not required.

The client application must ensure that the ID values do not collide, even across documents of different types. If collisions across types are possible in the application domain, then the client should unquify the ID, e.g. by prepending a unique digital prefix per type to the natural key.

email

Internet messages. Input files must adhere one of three file formats: either [RFC 5322](#) for email and RFCs [2045](#), [2046](#), and [2047](#) for MIME structures; Microsoft's [MS-OXMSG](#) (.msg) file format, or JSON files that match the

expected fields.

chat

Interactive chat sessions specified in a special [format](#) defined by Text Grid.

general

Document files such as plain text, or proprietary formats used by Microsoft Office and Adobe Acrobat.

Common Document Fields

All document types share a set of common fields that describe aspects of Text Grid processing, and do not directly correspond to data in the original source. The common fields may be optional in the context of specific operations; see individual operation descriptions for details. The common fields are not searchable unless otherwise noted.

type

String. The document type, either "email", "general", or "chat".

id

String. The document ID, stringified version of a 64 bit positive integer. The `id` field is searchable by using the [Ids Query](#)

source_uri

String. The URI that Text Grid uses to retrieve the source artifact to parse and populate the document. Max length is 1024 characters. This is only present in the source JSON and not in the returned document.

state

String. The document state. Possible values are "unprocessed", "processed", and "indexed".

language

String. An IETF code indicating the document's language. See the [Languages](#) section for more information, including supported values.

text

JSON structure. A structure that contains fields derived from the source document. All type-specific document fields are located within the `text` field.

Email Documents

Most email-specific fields have the same semantics as fields defined in [RFC 5322](#), although the field name and value syntax may be somewhat different.

```
{
  "type": "email",
  "id": "91238",
  "source_uri": "file://server/share/91238.eml",
  "state": "processed",
  "language": "en",
  "text": {
    "date": "2016-03-16T13:56:39.492Z",
    "from": ["Jon Smith <jsmith@example.com>"],
    "sender": "Mailer <mailagent@example.com>",
    "reply_to": ["<jsmith@example.com>"],
    "to": ["Alexis Webster Belle <abelle@recipient.com>"],
    "cc": ["instd@recipient.com", "Normandy98 nmd98@also.biz"],
    "bcc": ["mgr243@tangential.net"],
    "message_id": "<L15XFGGRED2QUBFTXOLZDHB0BWAJZGNMB@zlsvr22>",
    "in_reply_to": ["<OMWOQTGQD4FHB4ETXMNCPK3AAYHL2KU0B@zlsvr22>"],
    "references": ["<OMWOQTGQD4FHB4ETXMNCPK3AAYHL2KU0B@zlsvr22>"],
    "subject": "Re: Transaction questions",
    "body": "You passed the review."
  }
}
```

date

Time in ISO 8601 format. Value derived from the Origination Date field, "Date". Note that the date time format differs from that used in [RFC 5322](#).

from

String array. Values derived from the Originator "From" field. Note that [RFC 5322](#) specifies that multiple values are permitted for this field, although this may be counterintuitive.

sender

String. Value derived from the Originator "Sender" field.

reply_to

String array. Values derived from the Originator "Reply-To" field.

to

String array. Values derived from the Destination Address "To" field.

cc

String array. Values derived from the Destination Address "Cc" field.

bcc

String array. Values derived from the Destination Address "Bcc" field.

message_id

String. Value derived from the Identification "Message-Id" field.

in_reply_to

String array. Values derived from the Identification "In-Reply-To" field.

references

String array. Values derived from the Identification "References" field.

subject

String. Value derived from the Informational "Subject" field.

body

String. Value derived from the main body of the email.

Note Only the subject, body, and from fields are available for searching with a match query

Attachment Section

Note Attachment processing is deprecated in TG2

Chat Documents

The text structure of chat documents has three main sections: a set of members containing information at the chat session as a whole, a section describing each participant, and a section with the messages that comprise the chat.

Some chat document fields are required because Text Grid depends on them to enable searches.

```
{
  "type": "chat",
  "id": "1042",
  "source_uri": "file://server/share/20151025a.nxchat",
  "state": "processed",
  "language": "en",
  "text": {
    "chat_id": "d5bcdf6e-7654-4f9d-a95d-3764c84084cf",
    "excerpted": false,
    "start_time": "2015-10-25T22:34:51+00:00",
    "end_time": "2015-10-25T22:44:28+00:00",
    "participants": [
      {
        "participant_id": "TC0483",
        "name": "Stephen Arundhati",
```



```

        "display_name": "Stevarino",
        "role": "Agent"
      },
      {
        "participant_id": "0973741",
        "name": "John Smith",
        "role": "Customer"
      },
      {
        "participant_id": "info",
        "display_name": "Info",
        "role": "Automation"
      }
    ],
    "messages": [
      {
        "timestamp": "2015-10-25T22:34:48+07:00",
        "participant_id": "info",
        "content": "Please do not give credit card information during this chat."
      },
      {
        "timestamp": "2015-10-25T22:34:50+07:00",
        "participant_id": "TC0483",
        "content": "My modem stopped working!"
      },
      {
        "timestamp": "2015-10-25T22:35:03+07:00",
        "participant_id": "0973741",
        "content": "Hold the reset button down for twenty seconds."
      }
    ]
  }
}

```

Session Section

chat_id

String. Unique identifier for the interaction in the source system. This value is logically distinct from the document ID, which uniquely identifies the interaction within Text Grid. (However, a client application may choose to use the chat ID value as the Text Grid document ID if it is a 64 bit positive integer.)

excerpted

Boolean, defaults to false. True if this is excerpted from a long-lived room or channel, false otherwise.

excerpt_source_id

String. Identifies the long-lived chat room or channel from which this session was excerpted.

start_time, end_time

Time in ISO 8601 format. Timestamps for the chat as a whole. Not necessarily identical to the earliest and latest messages.

Participants Section

participants

Array of participant objects, one for each participant in the session.

participant_id

String. Identifies the participant in external systems outside the context of the chat. Also used to correlate chat messages back to the participant sending the message.

name

String. The participant's name as recorded in external systems. May differ from what other participants see in the chat session.

display_name

String. The participant's name as presented to other chat participants.

role

String. Identifies the participant's role to enable role-specific text search. The client application must define the set of meaningful values. In a call center environment, suggested values are Agent, Customer, and Automation.

Messages Section

messages

JSON object array. Message ordering is "natural" as defined by the external system. In most systems, the messages are ordered to follow the flow of conversation. But in some systems, participants may experience variations in presentation order due to race conditions. This may line up with timestamp ordering, but this is not guaranteed unless the external system does so.

timestamp

Time in ISO 8601 format. Timestamp of chat message as recorded by the source system. Note that this may differ from the precise local time the message was sent or was actually presented to other participants.

participant_id

String. Reference to element in the [participants](#) array indicating which participant sent the message.

content

String. Text content of the message.

General Documents

General documents have the simplest structure of all Text Grid types.

```
{
  "type": "general",
  "id": "91239",
  "source_uri": "file://server/share/note.txt",
  "state": "processed",
  "language": "en",
  "text": {
    "content": "Get eggs and milk!"
  }
}
```

content

String. Text content of the document.

Languages

Human languages require language-specific treatment in order to properly break down text content into individually indexable terms (i.e. *tokens*).

To perform the correct language-specific processing, Text Grid needs to know the language of the document being processed. The client specifies this in the document's `language` field as well as in search requests. Supported values are derived from IETF language tags as defined in [RFC 5646](https://tools.ietf.org/html/rfc5646). In most cases, the tags are two-letter primary language tags, but some values may include a region subtag. Language values must be exact matches; additional subtags or casing differences are not supported.

The language code "ALL" can be used in some cases to indicate that the search, get, or delete, needs to look in all the languages for the indicated document.

The following languages are fully supported:

- English: `en`
- French: `fr`
- Spanish: `es`

The following additional languages are available for experimentation, but they are not supported and do not benefit from

any testing at NICE. In particular, support for Chinese, Japanese, and Korean is expected to be particularly weak at this point.

- Arabic: [ar](#)
- Armenian: [hy](#)
- Basque: [eu](#)
- Bulgarian: [bg](#)
- Catalan: [ca](#)
- Chinese: [zh](#)
- Czech: [cs](#)
- Danish: [da](#)
- Dutch: [nl](#)
- Finnish: [fi](#)
- Galacian: [ga](#)
- German: [de](#)
- Greek: [el](#)
- Hindi: [hi](#)
- Hungarian: [hu](#)
- Indonesian: [id](#)
- Irish: [ga](#)
- Italian: [it](#)
- Japanese: [ja](#)
- Korean: [ko](#)
- Latvian: [lv](#)
- Lithuanian: [li](#)

- Norwegian: [no](#)
- Persian: [fa](#)
- Polish: [pl](#)
- Portugese (European): [pt](#)
- Portugese (Brazilian): [pt-BR](#)
- Romanian: [ro](#)
- Russian: [ru](#)
- Sorani: [ckb](#)
- Swedish: [sv](#)
- Turkish: [tr](#)
- Thai: [th](#)

Error Responses

Error response entities contain several categories of information. This information is contained in entity fields and in some cases as HTTP headers as well.

timestamp

The time the error occurred.

code

A well-known string signifying the category of error. The error code is intended for automated processing, but it is human-readable as well. It is also available in the [x-nx-error-code](#) HTTP header.

message

A human-readable string describing the error. Error messages are intended for administrator or technical support users, so client applications should not present error messages directly to end users. Error message content can change from release to release, so client applications should base error handling logic on the error code, not the error message.

resource_uri

A URI indicating a resource relevant to the error. In most cases, the error resource is the same as the HTTP request URI. But some errors point to a different resource. It is also available in the [x-nx-error-resource](#) HTTP

header.

exception

If the root cause of the error is an exception, this will provide the name of the exception.

errors

An abbreviated stack trace of the error that may help track down the problem. This may change from release to release, so client applications should base error handling on the error code, not the stack trace.

```
{
  "timestamp": "2019-07-10 10:31:18.929",
  "code": "not_found",
  "resource_uri": "http://localhost:28002/document",
  "exception": "TgEntityNotFoundException",
  "message": "failed getting GeneralDocument(super=DocumentInfo{id='11', sourceUri=file:/tmp,
state=unprocessed, language=es, documentType=general}) from path: file:/tmp",
  "errors": [
    <stacktrace>
  ],
}
```

Global Error Responses

The following error codes are applicable to all API operations. In most cases, the terminology and semantics are drawn from the HTTP 1.1 standard as defined in [RFC 2616](#).

unsupported_media_type

The request's content type is not set to application/json.

internal_server_error

An internal error occurred while processing request.

bad_request

The Requested URL or input entity was malformed.

request_entity_too_large

The input entity is too large for the server to accept.

service_unavailable

Text Grid is currently unable to handle the request due to a temporary overloading or maintenance of the server.

not_found

Text Grid could not locate the requested resource.

gone

The requested resource has been deleted, but Text Grid has an internal record of the resource having existed.

Security

Client requests are authenticated using the HTTP Basic scheme as defined in [RFC 7617](#). There are three types of users:

- admin - The administrative user has the ability to perform system maintenance activities.
- service - The service user is used internally by Text Grid for inter-service activities.
- user - An application user that performs typical activities such as indexing and searching.

Important: For optimum performance, client applications should send credentials preemptively; that is, do not wait to receive a 401 (Unauthorized) challenge to send credentials.

Reference: The *Administration Guide* describes how to manage users and update credentials.

HTTPS Support

As discussed above, Text Grid uses the HTTP Basic authentication scheme, which involves transmitting credentials in (base 64 encoded) cleartext. To encrypt credentials and other user data in transit, an administrator can configure Text Grid to use HTTPS for communication with client applications. Reference: The *Administration Guide* describes how configure Text Grid for encrypted communication.

Document Operations

Add/Update Document

Adds or updates a document to Text Grid for processing and indexing. Add/Update Document is simple to use, but production applications should use [Add/Update Document Batch](#) for increased efficiency.

See the [Get Document](#) operation for details about the entity structure. This section will review some special considerations for documents in the context of the Add Document operation.

The `id` field is required.

The `state` field is required. Valid values are "unprocessed" if the document has no `text` field, and "processed" if the

document is supplied with a `text` field.

If the new document includes a `text` element, the `source_uri` member will be ignored.

On success with a new document, the operation responds with 201 (Created). The response does not contain an entity, but this may change in a future release.

If a document with the specified ID already exists, the operation responds with 200. The response contains an entity.

This command can be used to change the contents of an existing document, which will force re-indexing of the document. The recommended procedure is to use Get Document to retrieve the data, make the desired changes, then use Update Document to push the changes into Text Grid. Note that the `id`, `type`, and `language` members are immutable and must be present. If the incoming values do not match, the document will not be properly updated and results are undefined.

Changing the document state has the following effects:

unprocessed

Document will be reprocessed and indexed, replacing the `text` if present.

processed

Document will be re-indexed with the existing `text`.

All other values

Invalid; update fails.

Request using PUT:

```
PUT /document
{
  "id": "91239",
  "type": "email",
  "source_uri": "file://server/share/91238.eml",
  "state": "unprocessed",
  "language": "en"
}
```

Request using PUT with pre-processed document:

```
PUT /document?action=add
{
  "id": "91239",
  "type": "general",
  "source_uri": "file://server/share/note.txt",
  "state": "processed",
  "language": "en",
  "text": {
    "content": "Get eggs and milk!"
  }
}
```



```
| }  
| }
```

Response when the document has not yet been added:

```
| HTTP/1.1 201 Created  
| Location: http://127.0.0.1:28002/document/91238
```

Response to updating an existing document:

```
| HTTP/1.1 200 OK  
| {  
|   "type": "general",  
|   "id": "91239",  
|   "state": "processed",  
|   "language": "en",  
|   "text": {  
|     "content": "Get eggs and cheese!"  
|   }  
| }
```

Add/Update Document Batch

Add or Update multiple documents to Text Grid for processing and indexing. This operation is more efficient than [Add/Update Document](#), but is also more complex to use.

Document additions in the request are not atomic with respect to the request; that is, in error conditions, some documents may fail while others succeed.

The 200 (OK) response code means only that the request was syntactically correct and acted upon. It does not imply that the documents were added successfully. Any 4xx or 5xx response code indicates an error, and the response entity contains a single error structure rather than result arrays.

The `action` field is required, and must be either 'add' or 'update'. In Text Grid 2.x and above, there is no difference between the add and update operation. Unlike Add Document, this operation uses POST rather than PUT, and the `id` field is required.

Elements of the returned `documents_succeeded` array containing the IDs of the added documents. Elements in the `documents_failed` array contain error information for documents that could not be added.

Request:

```
| POST /document?action=add  
| {  
|   "documents": [  
|     {  
|       "type": "email",  
|       "id": "91238",  
|       "source_uri": "file://server/share/91238.eml",  
|     }  
|   ]  
| }
```

```

    "state": "unprocessed",
    "language": "en"
  },
  {
    "type": "general",
    "id": "91239",
    "source_uri": "file://server/share/note.txt",
    "state": "processed",
    "language": "en",
    "text": {
      "content": "Get eggs and milk!"
    }
  },
  {
    "type": "unknown",
    "id": "91240",
    "source_uri": "file://server/share/badfile.zzz",
    "state": "processed",
    "language": "en"
  }
]
}

```

Response:

```

HTTP/1.1 200 OK
{
  "documents_succeeded": [
    {
      "id": "91238",
    },
    {
      "id": "91239",
    }
  ],
  "documents_failed": [
    {
      "code": "bad_request",
      "message": "Invalid document type 'unknown'",
      "id": "91240"
    }
  ]
}

```

Get Document

Returns content of the document, including its lifecycle state and any parsed text and metadata fields. Response entities are in the formats described in the [Document Types](#) section.

Get Document is simple to use, but [Get Document Batch](#) is a better choice for increased efficiency.

Request:

```
GET /document/<doctype>/<language>/<document_id>
```

Response (with processed document):

```
HTTP/1.1 200 OK
{
  "type": "email",
  "id": "91238",
  "state": "processed",
  "language": "en",
  "text": {
    "date": "2015-8-12T08:59:01+01:00",
    "from": ["Jon Smith <jsmith@example.com>"],
    "sender": "Mailer <mailagent@example.com>",
    "reply_to": ["<jsmith@example.com>"],
    "to": ["Alexis Webster Belle <abelle@recipient.com>"],
    "cc": ["instd@recipient.com", "Normandy98 nmd98@also.biz"],
    "bcc": ["mgr243@tangential.net"],
    "message_id": "<L15XFGGRED2QUBFTXOLZDHB0BWAJZGNMB@zlsvr22>",
    "in_reply_to": ["<OMWOQTGQD4FHB4ETXMNCPK3AAYHL2KU0B@zlsvr22>"],
    "references": ["<OMWOQTGQD4FHB4ETXMNCPK3AAYHL2KU0B@zlsvr22>"],
    "subject": "Re: Transaction questions",
    "body": "You passed the review."
  }
}
```

Get Document Batch

Returns the content of multiple documents. Compared to [Get Document](#), Get Document Batch is more efficient, but also more complex to use.

The `action` parameter is required and must be set to "get". Unlike Get Document, this operation uses POST rather than GET.

Elements of the returned `documents_succeeded` array contain the requested documents in the formats described in the [Document Types](#) section. Elements in the `documents_failed` array contain error information for documents that could not be retrieved.

Request:

```
POST /document?action=get
{
  "documents": [
    {
      "id": "91238",
      "type": "email",
      "language": "en"
    }
  ]
}
```

```

    {
      "id": "91239",
      "type": "general",
      "language": "en"
    },
    {
      "id": "91240",
      "type": "general",
      "language": "en"
    }
  ]
}

```

Response:

```

HTTP/1.1 200 OK
{
  "documents_succeeded": [
    {
      "type": "email",
      "id": "91238",
      "state": "processed",
      "language": "en",
      "text": {
        "date": "2015-8-12T08:59:01+01:00",
        "from": ["Jon Smith <jsmith@example.com>"],
        "sender": "Mailer <mailagent@example.com>",
        "reply_to": ["<jsmith@example.com>"],
        "to": ["Alexis Webster Belle <abelle@recipient.com>"],
        "cc": ["instd@recipient.com", "Normandy98 nmd98@also.biz"],
        "bcc": ["mgr243@tangential.net"],
        "message_id": "<L15XFGGRED2QUBFTXOLZDHB0BWAJZGNMB@zlsvr22>",
        "in_reply_to": ["<OMWOQTGQD4FHB4ETXMNCPK3AAYHL2KU0B@zlsvr22>"],
        "references": ["<OMWOQTGQD4FHB4ETXMNCPK3AAYHL2KU0B@zlsvr22>"],
        "subject": "Re: Transaction questions",
        "body": "You passed the review."
      }
    },
    {
      "type": "general",
      "id": "91239",
      "state": "processed",
      "language": "en",
      "text": {
        "content": "Get eggs and cheese!"
      }
    }
  ],
  "documents_failed": [
    {
      "code": "gone",
      "message": "The requested document has been deleted.",
      "id": "91240"
    }
  ]
}

```

```
|  }  
  ]  
}
```

Get Tokens

Returns the tokenized form of the specified fields for a given document. The output can be used as input into a word cloud creation algorithm.

Only documents in the *indexed* state are eligible for the Get Tokens operation. Attempting to retrieve tokens for a document in any other state will result in a *forbidden* error.

The *field* query parameter specifies the field names for which to return tokens. Multiple *field* parameters are permitted, and at least one is required.

In the response, the *fields* member contains one object per specified field; the member name is the name of the requested field. The field-specific objects contain a set of token-specific members; again, the member name is the value of the token.

Each token-specific object contains two members. The *term_freq* member is an integer describing the number of times the token appears in the document. The *tokens* array contains objects describing specific instances of the token in the document. Therefore, the *term_freq* value describes the length of the *tokens* array.

In elements of the *tokens* array, the *start_offset* and *end_offset* members describe the zero-indexed location of the token in terms of characters; the start is inclusive and the end is exclusive. The *position* member describes the instance's zero-indexed location in terms of token count.

Tokens for chat documents will include a *message_index* member, which identifies the zero-indexed ordinal position of the message within the chat session. The *position*, *start_offset* and *end_offset* members will be indexed from the beginning of the message within the chat session.

If the Get Tokens operation is used to retrieve tokens for array-valued fields (such as the *from* field in email documents), the values of members in the *tokens* array is unspecified. The *term_freq* value and the size of the *tokens* array will be correct, but the values of all *start_offset*, *end_offset*, and *position* members are unspecified and may change in future Text Grid releases.

Request:

```
| GET /document/<doctype>/<language>/<document_id>/tokens?field=<first_field>[&field=<second_field>]
```

Response (for email and general document types):

```
| HTTP/1.1 200 OK  
| {  
|   "type": "email",  
|   "id": "12345",  
|   "language": "en",
```

```

"fields": {
  "subject": {
    "a": {
      "term_freq": 1,
      "tokens": [
        {
          "end_offset": 1,
          "position": 0,
          "start_offset": 0
        }
      ]
    },
    "new": {
      "term_freq": 1,
      "tokens": [
        {
          "end_offset": 5,
          "position": 1,
          "start_offset": 2
        }
      ]
    }
  },
  "body": {
    "test": {
      "term_freq": 1,
      "tokens": [
        {
          "end_offset": 4,
          "position": 0,
          "start_offset": 0
        }
      ]
    }
  }
}

```

Response (for the chat document type):

```

HTTP/1.1 200 OK
{
  "type": "chat",
  "id": "12345",
  "language": "en",
  "fields": {
    "content": {
      "a": {
        "term_freq": 1,
        "tokens": [
          {
            "message_index": 0,
            "end_offset": 1,

```

```

        "position": 0,
        "start_offset": 0
      }
    ]
  },
  "new": {
    "term_freq": 1,
    "tokens": [
      {
        "message_index": 0,
        "end_offset": 5,
        "position": 1,
        "start_offset": 2
      }
    ]
  },
  "test": {
    "term_freq": 1,
    "tokens": [
      {
        "message_index": 1,
        "end_offset": 4,
        "position": 0,
        "start_offset": 0
      }
    ]
  }
}
}
}
}
}

```

Get Tokens Without Indexing a Document

Sometimes, one may want the tokenization of a document, but not need to actually store the document in the ElasticSearch database. This can be done with the following endpoint:

```

POST /document/tokens?field=<first_field>[&field=<second_field>]
{
  ...document as in the Add Document call...
}

```

Delete Document

Removes a document from Text Grid. Note that only resources within Text Grid are affected; in particular, the artifact originally referenced by the `source_uri` field is *not* deleted. Delete Document is simple to use, but in most production applications, [Delete Document Batch](#) should be used instead for efficiency.

Delete Document is a synchronous operation.

Request:

```
DELETE /document/<doctype>/<language>/<document_id>
```

Response:

```
HTTP/1.1 200 (OK)
```

Delete Document Batch

Removes multiple documents from Text Grid. This operation is more efficient, but also more complex to use, than Delete Document. See [Delete Document](#) for a description of the relationship between deleted documents and search results.

Document deletions in the request are not atomic with respect to the request; that is, in error conditions, some deletions may fail while others succeed.

The 200 (OK) response code means only that the request was syntactically correct and acted upon. It does not imply that the documents were deleted successfully. Any 4xx or 5xx response code indicates an error, and the response entity contains a single error structure rather than result arrays.

The `action` parameter is required and must be set to "delete". Unlike Delete Document, this operation uses POST rather than DELETE.

Elements of the returned `documents_succeeded` array contain the IDs of successfully deleted documents. Elements in the `documents_failed` array contain error information for documents that could not be deleted.

Request:

```
POST /document?action=delete
{
  "documents": [
    {
      "id": "91238",
      "type": "email",
      "language": "en"
    },
    {
      "id": "91239",
      "type": "general",
      "language": "en"
    },
    {
      "id": "91240",
      "type": "general",
      "language": "en"
    }
  ]
}
```


Response:

```
HTTP/1.1 200 OK
{
  "documents_succeeded": [
    {
      "id": "91238"
    },
    {
      "id": "91239"
    }
  ],
  "documents_failed": [
    {
      "code": "not_found",
      "message": "Document '91240' not found",
      "id": "91240"
    }
  ]
}
```

Search Operations

Search

Returns documents that meet the specified criteria. The request and response entities for Search are based on the Elasticsearch Search API and Query DSL, but with simplifications that make the operation easier to learn and use.

Search results may optionally describe specific regions of each document that correspond to the search criteria. These regions are called *highlights* because user interfaces often highlight them when displaying the document.

Search terms must be handled in a language-specific manner (see [Languages](#)). Therefore, all searches are language-specific, returning only those documents that were processed and indexed with the language specified in the search request.

Because Text Grid is designed to be used with large volumes of data, clients must be prepared for searches that return a large number of documents. There are two techniques for coping with large result volumes: One is to cap the result count with the `size` field, and the other is to receive paged results by specifying a value for the `scroll` field. See [Get Next Scroll](#) operation for more details on scrolling.

Relevance Scores

Search results contain a *relevance score* for each document found by the search. The relevance score is a measure of how interesting the document is likely to be to a human user, relative to the other documents found by the search. An application may want to present documents with a higher score higher in the list of results shown to the end user.

Relevance scores are computed using a technique commonly called *TF/IDF* relevance, which uses three primary inputs:

- *Term frequency*, a measure of how common the search term is in the document field where it was found. The more the term appears in the document, the higher its relevance.
- *Inverse document frequency*, a measure of how common the search term is across all indexed documents. Hits on common terms yield lower relevance scores than hits on uncommon terms.
- *Field length*. Hits in shorter fields (like an email subject versus an email body) yield higher relevance scores, essentially because a higher proportion of the field matches the search.

Relevance scores are relative and search-specific, intended to rank order documents within a given search. Score values from different searches—even searches with the same query—are not comparable.

Users of NICE's other products may confuse relevance scores with the *confidence scores* used by NICE phonetic technology. The two kinds of scores differ in important ways. Confidence scores express the likelihood that a given media file is a true match to the user's search; lower scores indicate a higher chance of a false alarm (a result that is not actually what the user was looking for). In contrast, the results of text searches are *guaranteed* to match the user's query, so false alarms are impossible. The relevance score attempts to anticipate a user's relative interest in one document over another, not the chance that the document actually matches the query.

Searching Chat Documents

Searches against chat documents can be performed at message or session levels. For example, consider the following chat transcript:

.....

Agent: Can I help with anything else?

Customer: No thank you.

Agent: Great, have a nice day!

.....

A search for "else" within four words of "great" will hit at the session level, because the two words are separated by only three words, the customer typing "No thank you." But it will not hit at message level because the message are evaluated individually, and the words of interest appear in separate messages.

Query Clauses

Text Grid supports many query clauses defined by the Elasticsearch Query DSL. This includes leaf clauses such as `ids` and `match`, and compound clauses such as `bool` and `not`. In addition, Text Grid adds a unique leaf clause, `nxtgsq`, that enables a query syntax based on the phonetic structured query language used in other NICE products. The `nxtgsq` clause can be freely combined with other clauses in compound structures.

Text Grid supports the Elasticsearch query clauses listed below. Developers unfamiliar with Elasticsearch should start with the `match`, `ids`, and `bool` queries.

Full text queries

- [Match Query](#)
- [Multi Match Query](#)

Term level queries

- [Term Query](#)
- [Terms Query](#)
- [Range Query](#)
- [Exists Query](#)
- [Missing Query](#)
- [Prefix Query](#)
- [Wildcard Query](#)
- [Regexp Query](#)
- [Fuzzy Query](#)
- [Ids Query](#)

Compound queries

- [Constant Score Query](#)
- [Bool Query](#)
- [Dis Max Query](#)
- [Boosting Query](#)

Sample Requests and Responses

Search email for messages that include "internet" and "down" within 10 words of each other.

```
POST /search
{
  "type": "email",
  "language": "en",
  "return_highlighting": true,
  "query": {
    "nxtgsq": {
```

```

    "field": "body",
    "sq": "and_10(\"internet\", \"down\")"
  }
}

```

A response would look as follows:

```

HTTP/1.1 200 OK
{
  "duration": 9,
  "timed_out": false,
  "hits": {
    "total": 1,
    "max_score": 2.4157481,
    "documents": [
      {
        "id": "1235678",
        "score": 2.4157481,
        "highlight": {
          "body": [ "10-18;29-33" ]
        }
      }
    ]
  }
}

```

Although neither language nor type is a required field, leaving one out is not usually the recommended usage. Since the field names differ amongst the different document types, and chats use hidden fields to represent inner structure, leaving out the doc type is rarely useful. Leaving out the language may occasionally be useful to search for trade names or some other word that is the same in multiple languages:

```

POST /search
{
  "type": "general",
  "return_highlighting": true,
  "query": {
    "nxtgsq": {
      "field": "content",
      "sq": "and_10(\"MYBRAND\", \"Modem\")"
    }
  }
}

```

One common scenario is that metadata in an external system can be used to narrow the search to a subset of documents. The subset can then be expressed to Text Grid via an `ids` clause, as shown in the example request below. The example uses a `bool` query to combine the `nxtgsq` and `ids` clauses. The `nxtgsq` clause is placed in `bool` 's `must` clause because we want the structured query to contribute to relevance scores in the results. The `ids` clause is placed in `bool` 's `filter` clause because `ids` should not contribute to the relevance score. (See Elasticsearch's [documentation](#) for more about filters.)

Note that in performance testing, it has been determined that extremely large filter id lists (> 10,000) show exponentially worse performance. If you need to filter a dataset for very large id sets, it is better to break the query into multiple queries, each with a smaller id list, rather than combining them into a single query. This is also necessary if you wish to retrieve a total number of hits greater than the **size** parameter (default 10,000). While the specific best id list size is dependent on multiple factors including the number of found documents, our testing indicates that a size greater than 10,000 is contraindicated, and a size of 1000 is faster than 5000 or 10000.

Note As of Text Grid 20.1.0, documents are grouped into indices by their ID in batches of 1 megabyte. This means that if your id list is in only 1 or 2 of the indices, the search will be quicker than if it covers all the indices of that type.

```
POST /search
{
  "type": "email",
  "language": "en",
  "return_highlighting": true,
  "query": {
    "bool": {
      "must": [
        {
          "nxtgsq": {
            "field": "body",
            "sq": "and_10(\\\"open\\\", \\\"account\\\")"
          }
        }
      ]
    },
    "filter": {
      "ids": {
        "values": [
          "12345678",
          "23456789",
          "567328867"
        ]
      }
    }
  }
}
```

Request Fields

type

A string value indicating the type of documents to search. Legal values are "email", "general", and "chat".

language

A string value indicating the document's language. See the [Languages](#) section for more information, including supported values.

chat_scope

Optional, defaults to "message". Describes the kind of chat search to execute. Valid values are "message" (the default) and "session".

scroll

Optional, defaults to no scrolling. The duration the scroll context should be kept alive. The duration is specified as an integer followed by a letter indicating a time unit. For example, "1m" would indicate one minute. Available time units include "m" for minute, "s" for second, and "ms" for millisecond.

size

Optional, defaults to 10000. If using a scroll search, defines the maximum number of document results to be returned in each batch of the scroll. If not using a scroll search, the maximum number of documents to be returned by the search. The default value of 10,000 is the maximum allowed.

query

'query' is required. An object describing the query to evaluate against the documents. Text Grid extends the Query DSL with a structured query syntax designed for compatibility with NICE phonetic analysis products. See the [Structured Queries](#) section for details of Text Grid's structured queries; see the Elasticsearch [documentation](#) for information about the Elasticsearch Query DSL.

return_highlighting

Optional, defaults to true. A Boolean value indicating whether to return highlighting information for hits. Disabling highlighting may improve search performance.

Response Fields

scroll_id

Present only in scrolled search results. The value should be passed in the subsequent [Get Next Scroll](#) operation.

duration

The wall clock time in milliseconds required to process the request.

timed_out

A Boolean value indicating whether the search timed out. If the value is true, it is possible that only partial results (if any) were returned.

hits

A JSON object encapsulating the search results.

total

The number of documents found in the search.

max_score

The maximum relevance score of the documents found by the search.

documents

A list of document objects found in the search. Documents are not returned in any particular order.

id

The ID of the document.

score

The relevance score of the document.

highlight

A JSON structure describing what regions of the document met the search criteria. The format of the structure is covered in the next section.

Highlight Data String Format

A highlight structure comprises a set of field names, each associated with an array of Strings. The array has exactly one element, a formatted string. (The highlight data is in an array rather than a scalar field because Elasticsearch supports multiple highlighting techniques, some of which may return multiple elements in the array. Text Grid's highlighting generates only a single array element, but the API retains Elasticsearch's structure for extensibility.)

The format of the data string depends on the type of document being searched. For email and general documents, the string is a semicolon-delimited sequence of character ranges that indicate the characters in the indexed field that matched the search. Character ranges are in the format `<min>-<max>`, where `<min>` is the zero-indexed beginning of the range (inclusive) and `<max>` is the end of the range (exclusive). For example, a search for "cat" in the text "The cat in the hat" would return highlight data "4-7", indicating that "cat" was found from character four (inclusive) to character seven (exclusive) of the text. Note that character offsets specifically refer to characters, not bytes; multibyte Unicode characters count as one character.

For chat searches, the highlight data string uses the format `<msg_id>:<min>-<max>`. The `<msg_id>` part indicates the zero-indexed ordinal position of the matching message within the chat session. Note that the position is based on *document order*, not on message timestamps. The `<min>` and `<max>` fields have the same meaning as in email and general documents.

Get Next Scroll

Retrieves the next batch of search results in the scroll and deletes the results associated with the previous `scroll_id` value. The response will contain a new `scroll_id` value which should be used in the next Get Next Scroll invocation.

When the returned `hits` array is empty, the search is completed. To conserve Text Grid resources, the client should invoke [Clear Scroll](#) in this situation.

scroll

String, required. The duration the scroll context should be kept alive after this invocation of Get Next Scroll. The duration is specified as an integer followed by a letter indicating a time unit. For example, "1m" would indicate one minute. Available time units include "m" for minute, "s" for second, and "ms" for millisecond.

scroll_id

String, required. The `scroll_id` value received in the response to the previous Search or Get Next Scroll call.

Request:

```
POST /search/scroll
{
  "scroll": "1m",
  "scroll_id": "7996315bcbbb43879da585981d7642dd"
}
```

Response:

```
HTTP/1.1 200 OK
{
  "scroll_id": "0378d918395443b186a847b0946ce0a4",
  "duration": 9,
  "timed_out": false,
  "hits": {
    "total": 1,
    "max_score": 2.4157481,
    "documents": [
      {
        "id": "1234598",
        "score": 2.4157481,
        "highlight": {
          "body": [ "10-18;29-33" ]
        }
      }
    ]
  }
}
```

Clear Scroll

Disposes of resources associated with a scroll. To maximize Text Grid efficiency, clients should invoke this after reaching the end of a search scroll (i.e. retrieving an empty `hits` array) or to cancel a search that is no longer relevant.

The part of the path after the last slash must contain the most recently returned `scroll_id` value for the scroll to be cleared.

Request:

```
DELETE /search/scroll/7996315bcbbb43879da585981d7642dd
```

Response:

```
HTTP/1.1 204 No Content
```

Other Operations

About

Retrieves version information for Text Grid and its major components.

Request:

```
GET /about
```

Response:

```
HTTP/1.1 200 OK
{
  "name": "NICE Text Grid",
  "version": "2.0.0.0",
  "components": {
    "Elasticsearch": "3.2.1",
    ...
  }
}
```

Response Fields

name

The name of the Text Grid application.

version

The version of the Text Grid application. The string format of this value may change from release to release.

components

A set of key:value pairs indicating some of the major components in Text Grid and their versions. Note that the exact component names are subject to change from release to release.

License

Retrieves license information for Text Grid.

Request:

```
GET /license
```

Response:

```
HTTP/1.1 200 OK
{
  "name": "NICE Text Grid",
  "license": {
    "Status": "active",
    "Type": "platinum",
    "Message": "Elasticsearch License will expire in 15 days",
    "ExpiryInDays": 15
  }
}
```

```
HTTP/1.1 200 OK
{
  "name": "NICE Text Grid",
  "license": {
    "Status": "expired",
    "Type": "platinum",
    "Message": "Elasticsearch License is expired",
    "ExpiryInDays": -15
  }
}
```

Response Fields

name

The name of the Text Grid application.

license

A set of key:value pairs indicating license information in Text Grid. Note that the exact field names are subject to change from release to release.

Search Metrics

Retrieves the size of the Elasticsearch index for auditing.

Request:

```
GET /metrics/search
```

Response:

```
HTTP/1.1 200 OK
{
  "index_size_bytes" : "8675309"
}
```

Response Fields

index_size_bytes

Size of the Elasticsearch index store in bytes.

Structured Query Validation

Identifies problems with structured query text, including syntax errors and warnings.

For valid queries, validation results include a set of query_expressions that indicate the structure of the query. See the Query Expressions section below for more details about how to interpret query expressions.

- A response of 200 (OK) does not imply that the structured query text is valid, only that validation was executed successfully.
- Line and column numbers refer to positions in the query text as submitted in the request, not in the response entity.
- The line and column elements are optional, and may not appear with some error or warning codes.

Request:

```
POST /structuredqueryvalidation
{
  "type": "chat",
  "language": "en",
  "field": "content",
  "query": "AND_5(\"service\", \"blackberry\")"
}
```

Response:

```
HTTP/1.1 200 OK
```

```

{
  "query_expressions" : [
    {
      "arguments" : [
        {
          "query_expression_id" : 2
        },
        {
          "query_expression_id" : 3
        }
      ],
      "id" : 1,
      "is_used" : true,
      "name" : "#0",
      "operator" : "AND",
      "operator_column" : 1,
      "operator_line" : 1,
      "used" : true,
      "window" : 5
    },
    {
      "id" : 2,
      "is_used" : true,
      "operator" : "TERM",
      "operator_column" : 7,
      "operator_line" : 1,
      "parent_expression_ids" : [
        1
      ],
      "term" : "service",
      "used" : true
    },
    {
      "id" : 3,
      "is_used" : true,
      "operator" : "TERM",
      "operator_column" : 17,
      "operator_line" : 1,
      "parent_expression_ids" : [
        1
      ],
      "term" : "blackberry",
      "used" : true
    }
  ],
  "query_text" : "AND_5(\"service\", \"blackberry\")",
  "valid" : true
}

```

Response with errors and warnings:

```

HTTP/1.1 200 OK
{

```

```

    "query_text" : "r=\"broken\"\\nAND_5(\"service\", \"blackberry\"",
    "syntax_errors" : [
      {
        "column" : 6,
        "error_code" : "UnmatchedParenthesis",
        "line" : 2
      }
    ],
    "syntax_warnings" : [
      {
        "column" : 3,
        "line" : 1,
        "warning_code" : "UnusedExpression"
      }
    ],
    "valid" : false
  }

```

Syntax Warning and Error Codes

Warnings:

- FuzzinessTooLarge
- FuzzyExpansionsTooLarge
- UnusedExpression

Errors:

- DuplicateExpressionName
- IllegalFuzzinessAndWildcards
- IncompleteExpression
- InvalidArgument
- InvalidCountParameter
- InvalidExpressionName
- InvalidFuzziness
- InvalidMacro
- InvalidOperator

- InvalidWindowParameter
- InvalidWeight
- MissingArgumentDelimiter
- MissingArgumentList
- MissingCountParameter
- MissingOperator
- MissingWindowParameter
- NoExpressionsFound
- NoSearchableText
- ReferencedExpressionNotDefined
- SyntaxError
- TokensAfterExpression
- TooFewArguments
- TooManyArguments
- UnmatchedParenthesis
- UnmatchedQuote
- WeightNotAllowed

Query Expressions

The set of query_expression elements for a given structured query are the query's **parse tree**; that is, how the search engine internally interprets the query. The response entity presents the query expressions as a flat list, but expressions contain references to one another that describe a treelike structure. Query expressions contain the following elements:

id

Identifies the query expression if it is referenced as an argument of another query expression.

name

The name assigned to the expression in the query text. If no name was supplied, the search engine generates one.

name_line

The one-indexed line number of the expression name in the original structured query text. Omitted if the expression does not have a name.

name_column

The one-indexed column number of the expression name in the original structured query text. Omitted if the expression does not have a name.

operator

Identifies the kind of expression. Refer to the Structured Query Language section for details about the supported operators.

operator_line

The one-indexed line number of the operator in the original structured query text. This member may be omitted in some rare cases.

operator_column

The one-indexed column number of the operator in the original structured query text. This member may be omitted in rare cases.

window

Describes the term proximity basis for operators that use one. Present only for AND, ANDNOT, SUBSET, BEFORE, FIRST, and LAST operators.

count

Parameter for the SUBSET operator; omitted for all other operators.

arguments

Identifies a query expression that is an input into this one. The document order of argument elements implies their order in the operator.

query_expression_id

Corresponds to the id attribute of the referenced query_expressions element.

weight

The relative contribution of the argument.

parent_expression_ids

A list of ids corresponding to the id attribute of an expression that consumes this expression as an argument. A given

expression may have zero or more parent expressions.

is_used

True if the expression contributes to query execution; false otherwise.

Chat File Format

Chat sessions lack a vendor-neutral standard file format along the lines of [RFC 5322](#) for email. Therefore, Text Grid defines its own file format to which incoming chat sessions must adhere. The format is essentially identical to the structure inside the `text` field in Text Grid [Chat Documents](#).

Chat files use the JSON format as defined in [RFC 7159](#). Producers should note in particular that valid character encodings are limited to those referenced in the RFC: UTF-8, UTF-16, and UTF-32. Byte order markers are not permitted.

Chat files and chat documents are structurally identical, but their data may differ for a given chat. Chat files may have HTML markup in message `content` fields. When Text Grid reads the file, it strips any markup it finds, so the `content` fields in Text Grid will not have markup.

The format is designed to express discrete sessions with a beginning and end time. Data from long-lived chat rooms should be modeled as discrete excerpts from the room.

By convention, Text Grid chat session files should have either a `.nxchat` or `.json` extension.

```
{
  "chat_id": "d5bcdf6e-7654-4f9d-a95d-3764c84084cf",
  "excerpted": false,
  "start_time": "2015-10-25T22:34:51+00:00",
  "end_time": "2015-10-25T22:44:28+00:00",
  "participants": [
    {
      "participant_id": "TC0483",
      "name": "Stephen Arundhati",
      "display_name": "Stevarino",
      "role": "Agent"
    },
    {
      "participant_id": "0973741",
      "name": "John Smith",
      "role": "Customer"
    },
    {
      "participant_id": "info",
      "display_name": "Info",
      "role": "Automation"
    }
  ]
},
```



```
"messages": [  
  {  
    "timestamp": "2015-10-25T22:34:48+07:00",  
    "participant_id": "info",  
    "content": "Please do not give credit card information during this chat."  
  },  
  {  
    "timestamp": "2015-10-25T22:34:50+07:00",  
    "participant_id": "TC0483",  
    "content": "My modem stopped working!"  
  },  
  {  
    "timestamp": "2015-10-25T22:35:03+07:00",  
    "participant_id": "0973741",  
    "content": "Hold the reset button down for twenty seconds."  
  }  
]
```

Structured Queries

Text Grid extends the Elasticsearch Query DSL with a query type that lets users express searches for very precise combinations of words and the positions between the words. The syntax is derived from the phonetic structured query language used by other NICE products such as Interaction Analytics and Search Grid. The intent is to minimize the training required for an analyst user to create and manage queries for phonetic and text content.

Structurally, Text Grid structured queries have two levels: A *JSON envelope* that links the query to the Elasticsearch Query DSL, and *structured query language* that is symmetric to NICE's phonetic structured query language. The structured query language component is stored as a string member in the JSON envelope.

Fuzziness

Fuzzy searches permit searching for words that are misspelled or have a typo in the target documents. Fuzzy searches work by expanding search terms to multiple terms that are variations on the original term. The degree of variation is called the *fuzziness factor*. Specifically, the fuzziness factor is the number of single-letter changes that can be made to a word to produce another word. A change can be an addition, deletion, or substitution of an individual letter. For example, *fox* to *fax* requires one change, substituting the letter *a* for the letter *o*.

Usually, only fuzziness factors from zero to two produce meaningful results; higher values yield too many false positives. A fuzziness factor of zero makes the search match the word exactly. Setting the fuzziness factor to "AUTO" lets the system derive a fuzziness factor based on the length of the word.

Each word in the phrase will be expanded to as many other words as possible during the search, which may reduce search performance. To keep search performance under control, the client can specify a *max expansion* count that constrains the number of term variations used.

Clients may also specify a number of characters at the beginning of the search term that are not subject to variation; this number is called a *prefix length*. For example, using a fuzziness factor of one and a prefix length of two, *fox* can be expanded to *for*, and *fop*, but not *fax*, *fix*, *box* or *lox*.

JSON Envelope

Example search request entity:

```
{
  "type": "email",
  "return_highlighting": true,
  "language": "en",
  "query": {
    "nxtgsq": {
      "field": "content",
      "sq": "and_10(\"internet\"~auto, \"down\"~1,10,2)"
    }
  }
}
```

field

String, required. Name of the field to search. Legal values are members of the documents' `text` field; for example, "body" and "subject" for email,

sq

String, required. The structured query itself, expressed in Text Grid's structured query language, described in the next section.

Structured Query Language

This section describes the syntax of the structured query language.

Comments

The semicolon is used to indicate that the remainder of the line is a comment and should not be processed.

Named Expressions

A named expression is any line in the Query file that is not a comment must be a statement of the form:

```
Name=<Expression>
```

The name expression can be any sequence of alphabetic characters, numeric characters or the underscore character (a-z, A-Z, 0-9, _). Names are not case-sensitive.

Phrase Expression

A phrase expression is an expression that represents a text search for a phrase. Here is an example of a named phrase expression:

```
| MyPhrase="Nexidia"
```

Fuzzy Phrases

A phrase expression may include a fuzziness specification, which will apply fuzziness to each term in the phrase. It is appended immediately after a quoted phrase using the following format. If no fuzziness specification is provided, the search will match *exactly* the terms in the phrase.

```
| ~<fuzziness>,<max_expansions>,<prefix_length>
```

fuzziness

Integer or "AUTO" (case-insensitive), defaults to AUTO. If set to an integer value, the fuzziness factor to apply to all terms in the phrase. If set to AUTO, the system will derive a fuzziness factor for each term based on its length.

max_expansions

Integer, defaults to 50. The maximum number of variations of each search term to use during the search. Smaller values may increase performance, and larger values may increase recall.

prefix_length

Integer, defaults to 2. The number of characters at the beginning of each search term that are not subject to change in fuzzy searches. Specifying a value zero may be legal, depending on the settings, but may dramatically reduce search performance.

Any parameter after the "~" may be omitted and the default value(s) will be used. However, if a parameter is omitted, all subsequent parameters must also be omitted. For instance, the following are all equivalent:

- ~
- ~AUTO
- ~AUTO,50
- ~AUTO,50,2

Here is an example of a fuzzy phrase expression:

```
| "Madam I am Adam"~AUTO,50,2
```

Operator Expression

An operator expression specifies a relationship among its arguments that must be satisfied.

AND

The arguments to this operator must all occur within a specified interval of words, or optionally, somewhere within the entire file.

OR

At least one of the arguments to this operator must occur.

ANDNOT

The second and subsequent arguments to this operator must not occur within a specified interval of words, or optionally, somewhere within the entire file, before or after an occurrence of the first argument.

SUBSET

At least the specified number of the arguments to this operator must occur within a specified interval of each other.

BEFORE

The first argument to this operator must occur within a specified interval of words, or optionally, somewhere within the entire file, before an occurrence of the second argument.

FIRST

The argument to this operator must occur within a specified interval from the beginning of the file.

LAST

The argument to this operator must occur within a specified interval from the end of the file.

NOTFIRST

The argument to this operator must occur outside a specified interval from the beginning of the file.

NOTLAST

The argument to this operator must occur outside a specified interval from the end of the file.

NOTLIKE

This operator is used in phonetic structured queries, but is not available for text searches.

Operator Arguments

Here is an example of an operator expression:

```
MyOperator=OR("L A", "Los Angeles")
```

This example is a named expression, MyOperator, that uses the OR operator to find occurrences of its arguments: "L A" and "Los Angeles."

In this example, the arguments in parentheses are unnamed phrase expressions. Any argument to an operator can be an unnamed phrase expression, an unnamed operator expression, or the name of a previously declared phrase or operator named expression.

As an example of this, Expression1 and Expression2 below are equivalent:

```
Example1=AND_60("This", OR("That", "The Other"))
Example2a=OR("That", "The Other")
Example2=AND_60("This", Example2a)
```

Operator Parameters

Most operators require one or more parameters in addition to their arguments. For example, the [AND](#) operator takes a parameter to indicate how close together its arguments must occur. These parameters are appended to the operator, separated by an underscore character, as follows:

```
ParamExample=AND_60("This", "That")
```

In this example, "This" and "That" must occur within 60 words of each other.

The [SUBSET](#) operator takes two parameters:

```
TwoParams=SUBSET_2_60("This", "That", "The Other")
```

In this example, the [SUBSET](#) operator will search for two of the given arguments to appear within 60 words of each other.

Like [OR](#), the operators [AND](#), [ANDNOT](#), and [BEFORE](#) can be used without specifying a window. In this case, they operate over the entire document being searched. You must be particularly careful when using this mode as it is very easy to get matches to component occurrences in completely unrelated parts of the document.

Line Continuation

Sometimes an expression may be too long to easily fit on a single line. In this case, the line-continuation character, backslash, can be used to indicate that the following line should be treated as an extension of the current one. For example: Sometimes an expression may be too long to easily fit on a single line.

```
ContinuedLine=OR("This", \
  "That", \
  "The Other")
```

Spacing and Casing

Whitespace and character casing is ignored. Thus, the following are equivalent:

```
Example=AND_60("This","That")
```

```
example = and_60 ("this", "that")
```

Output

The result of a query is the result of the last expression in the file. Any expressions, named or unnamed, that do not affect the last expression are not processed.

Placeholders

Placeholders are used to represent formatted data. Placeholders can be used anywhere a phrase is used, and can even be used within a phrase. Available placeholders include:

%ssn%

Matches a United States social security number in either `###-##-####` or `#####` format.

%money%

Matches money in the form `<CURCHAR>##, ##.##, <CURCODE> ##, <CURCODE> ##.##, ## <CURCODE>`, and `##.## <CURCODE>`, where `<CURCHAR>` is a Unicode 8.0 currency character (such as "\$") and `<CURCODE>` is an ISO currency code (such as "USD").

%ccn%

Matches credit card numbers in one of three patterns: a sequence of 12 to 19 digits, a pattern of 4-4-4-4 digits, or a pattern of 4-6-5 digits. Patterns can be separated by spaces or en dashes. The final digit must be a valid checksum according to the Luhn algorithm.

%usrn%

matches '@username' as used in social media like Twitter and Slack.

%hashtag%

matches '#hashtag' as used in social media.

%emoji%

matches specific emojis.

Emoticons and Emojis

Text Grid supports unicode emojis as a distinct token type that can be used in indexing and search (for the complete list,

see <http://unicode.org/Public/emoji/11.0/emoji-data.txt>). Also, a commonly used set of emoticons are treated the same as their graphical equivalent. For example, searching for ":" and "?" will return documents with either version.

Notably, emoji synonyms are not supported at this time. Therefore, documents with content: "Get some ?", "Get some pizza", "Give me a slice", will not turn up at the same time with a "fuzzy" search by "?" or "pizza". In other words, emoji related searches need be literal.

As of 12.4.2, there is a property (convertTextToEmoticon) in the application.yml file which if true will convert certain character sequences into their equivalent emoticon. Before 12.4.2, this behavior was always true. In order to maximize backwards compatibility, this property will be set to false on fresh installs of 12.4.2 and above (including migrations from TG 1.1.x), but will retain whatever behavior was already present on any upgrade.

Source URI Support

Document source URIs provided to Text Grid via the `source_uri` field must conform to the URI standard as specified in [RFC 2396](#), amended by [RFC 2732](#). The "file" and "http" schemes are supported. (Note in particular that the "https" scheme is not currently supported.)

When using the "file" schema, one must pay careful attention to the operating systems in use on the client application and in Text Grid itself. A file URI that can be accessed from a Windows server cannot be accessed from a Linux server, and vice versa.

File System Source URIs - Windows

File URIs should be constructed from Microsoft Windows Uniform Naming Convention (UNC) paths with an explicit server name. The following guidelines illustrate the conversion rules:

Convert paths with backslashes to forward slashes.

```
UNC:  \\myserver\myshare\mydoc.txt
URI:  file://myserver/myshare/mydoc.txt
```

Convert reserved URI characters using percent encoding.

```
UNC:  \\myserver\myshare\my#doc.txt
URI:  file://myserver/myshare/my%23doc.txt
```

Convert spaces using percent encoding.

```
UNC:  \\myserver\myshare\my doc.txt
URI:  file://myserver/myshare/my%20doc.txt
```

Convert literal percent characters using percent encoding.

```
UNC:  \\myserver\myshare\my%23doc.txt
URI:  file://myserver/myshare/my%2523doc.txt
```

For more information about expressing UNC paths as URIs, see [this MSDN article](#).

For compatibility, Text Grid accepts UNC path URIs in the format used by the Java framework's `File.toURI()` method. This method converts UNC paths by leaving the authority component empty and incorporating the server name and leading double slashes in the path component. As a result, four forward slashes appear after the scheme.

```
UNC:  \\myserver\myshare\mydoc.txt
URI:  file:///myserver/myshare/mydoc.txt
```

Java's `Path.toUri()` method produces a more traditional URI structure with only two leading slashes.

To support single-server installations, Text Grid accepts URIs constructed from local paths without an explicit server name. However, local URIs are discouraged because they do not support access in a distributed environment. The following examples illustrate supported forms of local path URIs:

```
Explicit "localhost" authority:  file://localhost/share/mydoc.txt
Implied "localhost" authority:  file:///folder/mydoc.txt
Windows drive letter (option 1): file:/c:/folder/mydoc.txt
Windows drive letter (option 2): file:///c:/folder/mydoc.txt
```

File URIs with a single leading slash but no drive letter will produce a relative path. Because Text Grid services are unlikely to use the same current directory as the client application, such URIs are discouraged, although they are considered valid:

```
Discouraged:  file:/discouraged/example/mydoc.txt
```

File System Source URIs - Linux

Common practice on Linux systems is to mount remote filesystems on the local filesystem hierarchy (usually under `/mnt`). Therefore, it is essential that a distributed Text Grid deployment have a common mountpoint policy across all servers, at least across those that access source documents. For example, if "myshare" is mounted as `/mnt/myshare` on one Text Grid server, it should be mounted as `/mnt/myshare` on *all* Text Grid servers; also, the client application must be aware of the mountpoint policy in order to construct usable document source URIs.

Linux file URIs must not contain an authority component, and therefore should have exactly three leading slashes. URIs with two leading slashes are considered invalid because they are usually the result of a .NET client supplying a UNC path, which Linux servers cannot access. Due to the "forgiving" nature of parsing frameworks, URIs with leading slash counts other than two or three will produce unspecified results.

```
Valid:      file:///mnt/myshare/mydoc.txt
Invalid:    file://myserver/myshare/mydoc.txt
Unspecified: file:/mnt/myshare/mydoc.txt
Unspecified: file:///mnt/myshare/mydoc.txt
```

The following conversion rules illustrate how to convert Linux file paths to URIs:

Convert reserved URI characters using percent encoding.


```
Path: /mnt/myshare/my#doc.txt
URI: file:///mnt/myshare/my%23doc.txt
```

Convert spaces using percent encoding.

```
Path: /mnt/myshare/my doc.txt
URI: file:///mnt/myshare/my%20doc.txt
```

Convert literal percent characters using percent encoding.

```
Path: /mnt/myshare/my%23doc.txt
URI: file:///mnt/myshare/my%2523doc.txt
```

File System Source URIs - All Platforms

Construction of invalid file URIs is a common source of integration problems. Most programming frameworks offer functions to convert file paths to URIs. Client application developers should use their framework's routines rather than write conversions from scratch.

Note to .NET users: Some .NET versions incorrectly handle file paths that contain square bracket characters ("[" and "]"). The framework routines generate an invalid URI with unescaped square brackets, and Text Grid will reject it. The recommended solution is to avoid placing square brackets in file paths. If this is not feasible, then the client application may require custom URI generation code.

The code snippets below may serve as a guide for the URI encoding step.

Create a URI string given a file path name in C#:

```
// See above note on paths with square brackets in .NET
Uri uri = new Uri(filePathname, UriKind.Absolute);
// Do NOT use Uri.ToString()!
string uriEncodedText = uri.AbsoluteUri;
```

Create a URI string given a file path name in Java:

```
File file = new File(filePathname);
URI uri = file.toURI();
String uriEncodedText = uri.toString();
```

Create a file path name from the URI-encoded string in C#:

```
Uri uri = new Uri(uriEncodedText, UriKind.Absolute);
string filePathname = uri.LocalPath;
```

Create a file path name from the URI-encoded string in Java:

```
URI uri = new URI(uriEncodedText);
File file = new File(uri);
String filePathname = file.getAbsolutePath();
```

HTTP Source URIs

Although this Guide usually describes Text Grid as a *server* application, Text Grid can also act as a *client*, downloading source documents from a web server for processing. HTTP-based source document access can support integration in environments where accessing source documents via the file system is not viable. One example is a deployment that uses mixed Windows and Linux operating systems.

To use HTTP source document access, the client application adds a document with a `source_uri` value that satisfies the following criteria:

- The scheme must be `http` (not `https`).
- The URI must be reachable from all Text Grid servers.
- Port and query parts are accepted.
- Userinfo and fragment parts are accepted but ignored. Be aware that Text Grid may write userinfo content to log files in cleartext.

■ Example: `http://server:8080/file.txt?site=atl#intro`

Text Grid's HTTP requests for source documents have the following characteristics:

- Each request is an HTTP GET operation.
- The `Accept` header is omitted.
- Text Grid issues requests for source documents when it is ready to process the document. Depending on number of documents to be processed, this could be hours or even days after the client application added the document.
- Text Grid may issue many concurrent requests. Consult the Installation and Configuration Guide for details, but in general, the peak concurrency level is equal to the total number of logical CPUs on all Text Grid servers.

The following characteristics apply to the web server's responses:

- Text Grid follows 301, 302, and 307 redirection responses. However, only the initial URI will contribute to file name determination (see below).
- Response codes in the 3xx, 4xx, or 5xx families (except for specific 3xx codes listed above) will cause Text Grid to abandon processing and place the document in the `processing_failed` state.
- Specifying a well-known `Content-Type` header value may help Text Grid recognize the entity type and facilitate extracting text from it.

Text Grid can also use the filename of the returned entity to facilitate parsing. To ascertain the filename, the system checks the following sources in order, using the first value that it finds:

1. The `Filename` parameter of the `Content-Disposition` header, as specified by [RFC 6266](#).
2. The `name` component of `Content-Type` header, in the grammar specified by [RFC 2045](#).
3. The last element of the path component of the URI, as specified by [RFC 2396](#)

Changes From Text Grid 1.x to Text Grid 2.x

Document Flow

In Text Grid 2.x and above, the responsibility for the System of Record, and for Parsing documents has been moved to the application. As such, if a database needs to be re-created, it must be done by re-ingesting the documents by the application rather than from internal state in Text Grid.

Get Tokens Support for artificial documents

If tokenization is wanted on a document that will never be searched, support has been added to be able to get tokenization without storing the document ([Get Tokens Without Indexing a Document](#))

Synchronous CRUD Calls

In Text Grid 1.x, CRUD calls were all asynchronous. These calls are now synchronous.

Notifications

Since Text Grid 2.x and above use synchronous ingest, instead of asynchronous ingest, the Text Grid 1.x notification mechanism has been eliminated. As such, the `enable_notifications` field has been eliminated.

Document Operations

id length

`id` was limited to 1024 in TG 1.1. However, Elasticsearch limits ids to 512 bytes long. This means that strings that are close to the 512 limit in characters, but require additional bytes to be stored in UTF-8 encoding, will be prohibited.

id values

`id` is normally a stringified version of a 64 bit positive integer as of 20.1.0. Any ids that did not fit this description (negative numbers, numbers greater than 64 bits, non numeric strings) will still work for now, but their use is deprecated. In future releases, these ids may be prohibited. For now, the logs will warn about any such ids that are added to the system, as well as providing warnings on the initial upgrade to a release that has this restriction.

Add and Update (location of id field)

In Text Grid 1.x, for [Add Document](#) and [Update Document](#) the `id` field was passed as part of the URI, but also could be passed as part of the request body. This led to confusion as to which version should be used if they differed. In 2.x, the `id` must be passed as part of the request body.

Add and Update have been merged.

In Text Grid 1.x, the Add and Update (both Document and Document Batch) were unique calls with different behavior. In Text Grid 2.x, the two calls are identical and thus either action parameter works.

Clear Scroll

The Clear Scroll operation now passes the scroll id in the path of the DELETE url instead of in a JSON body.

Additional Required Parameters

For API calls that previously identified existing document just by `id`, `doctype` and `language` have been added as part of the identification of where the document is stored.

Thus, for [Get Document](#), [Delete Document](#), and [Get Tokens](#), the `<doctype>` and `<language>` are now part of the request URL.

For [Get Document Batch](#) and [Delete Document Batch](#), `doctype` and `language` are now required elements in the JSON.

Note that ALL can be used for the language type in order to search all the languages for the requested document.

source_uri field

The `source_uri` field is not stored in the Elasticsearch database, nor is it returned on a `GetDocument` call.

Email files

Attachments are not supported in Text Grid 2.x and above.

Search Wildcards

In Text Grid 2.x, the `language` and `doctype` are not required fields for search. This allows someone to search for a phrase in email, chat, and document, or in multiple languages, with a single search.

In addition, if you wish to do a `Get Document`, `Get Tokens`, `Delete Document`, or `Delete Document Batch`, you can use the `ALL` language code to mean to search all the languages.

Note that this requires that you never use the same ID for documents in multiple languages. This is NOT enforced on adding documents, so this is up to the user to program correctly.

Get Document Status Batch Has Been Eliminated

There is no longer a special command to get all the documents in the database. This can easily be done with the following command.

```
POST /search?scroll=1m
{
  "query": {
    "match_all": { }
  }
}
```

Because no Document Type or Language are specified, all documents will be returned.

Support for Stemming Search has been eliminated

No special stemming support is included in Text Grid 12.4.

Support for some chat info fields has been removed

ip_address, *geo_location*, and *geo_coordinates* support has been removed.

Query Validation is more restrictive on Poison Queries by Default

The default `sqValidation` and search parameters discourage poison queries by requiring at least a 2 for the prefix for both wildcards and fuzziness (see Admin Guide for more details). This was true for the most recent clean installs of TG 1.1.x, but not for most of TG 1.1's releases.